

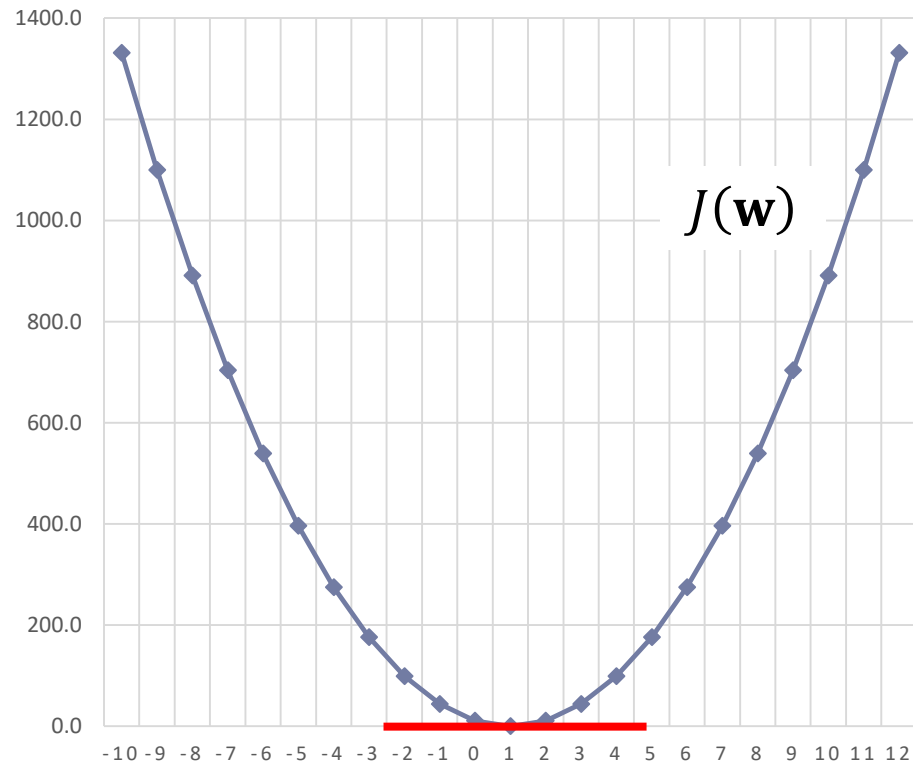
라이브러리를 활용한 선형 회귀

김성영교수
국립금오공과대학교
컴퓨터공학부

학습 내용

- 라이브러리를 활용하여 선형 회귀를 수행할 수 있다.

Normal equation



분석적analytically으로
 w 에 대한 최적 값 계산

Ordinary Least Squares (OLS) (1)

- 결정론적 회귀 분석 방법

- 단순히 독립 변수 x 에 대해 대응하는 종속 변수 y 를 계산하는 함수를 만드는 과정

- 미분을 통해 잔차제곱합(RSS: Residual Sum of Squares)를 최소화하는 파라미터 결정

- 잔차 벡터 (residual vector) $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \mathbf{X}\mathbf{w}$

$\mathbf{w}$ 는 $n+1$ 차원 벡터
 $\mathbf{X}$ 는 m 개 샘플을 갖는 $(m \times (n+1))$ 행렬

- 잔차 제곱합 (residual sum of squares)

$$\mathbf{e}^T \mathbf{e} = (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{y}^T \mathbf{y} - 2\mathbf{w}^T \mathbf{X}^T \mathbf{y} + \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w}$$

- 잔차 제곱합의 기울기 벡터

$$\frac{d\mathbf{e}^T \mathbf{e}}{d\mathbf{w}} = -2\mathbf{X}^T \mathbf{y} + 2\mathbf{X}^T \mathbf{X} \mathbf{w}$$

Ordinary Least Squares (OLS) (2)

- 최적화 조건

$$\frac{d\mathbf{e}^T\mathbf{e}}{d\mathbf{w}} = 0$$

$$\frac{d\mathbf{e}^T\mathbf{e}}{d\mathbf{w}} = -2\mathbf{X}^T\mathbf{y} + 2\mathbf{X}^T\mathbf{X}\mathbf{w}$$

$$\mathbf{X}^T\mathbf{X}\mathbf{w} = \mathbf{X}^T\mathbf{y}$$

Normal Equation

$$\mathbf{w} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

The corresponding solution $\mathbf{w}$ to the normal equation is called **ordinary least squares** (OLS) solution

With larger D(degree), N(samples) or higher order regression (instead of linear) make it highly challenging to derive $\mathbf{w}$ OLS

NumPy를 이용한 선형 회귀 (1)

```
from sklearn.datasets import make_regression
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits import mplot3d

# 학습 데이터 생성
bias = 100
X0, y, w = make_regression( (100×2) 행렬 생성
                             n_samples=100, n_features=2, bias=bias,
                             noise=10, coef=True, random_state=1)

# feature augmentation
X = np.hstack([np.ones((X0.shape[0], 1)), X0])

print("이론적 관계:")
print("y = %.2f + %.2f*x1 + %.2f*x2" % (bias, w[0], w[1]))
w = np.linalg.inv(X.T @ X) @ X.T @ y
print("추정한 관계:")
print("y = %.2f + %.2f*x1 + %.2f*x2" % (w[0], w[1], w[2]))
```

$$(w_1 \quad w_2 \quad w_3) \cdot \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = w_1x_1 + w_2x_2 + w_3x_3$$



feature augmentation

$$(b \quad w_1 \quad w_2 \quad w_3) \cdot \begin{pmatrix} 1 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} = b + w_1x_1 + w_2x_2 + w_3x_3$$

$$H(X) = X\mathbf{w}$$

$$\mathbf{w} = (X^T X)^{-1} X^T \mathbf{y}$$

NumPy를 이용한 선형 회귀 (2)

```
# 그래프 출력
X1_new = np.linspace(np.min(X0[:,0]), np.max(X0[:,0]), 100)
X2_new = np.linspace(np.min(X0[:,1]), np.max(X0[:,1]), 100)
m, n = np.meshgrid(X1_new, X2_new)

ax = plt.axes(projection='3d')
ax.set_xlabel('x1')
ax.set_ylabel('x2')
ax.set_zlabel('Score')
ax.scatter(X0[:,0], X0[:,1], y)

z = m*w[1] + n*w[2] + w[0]
ax.plot_surface(m, n, z, rstride=1, cstride=1, color='none',
alpha=0.4)
plt.show()
```

NumPy를 이용한 선형 회귀 (3)

이론적 관계:

$$y = 100.00 + 32.27 \cdot x_1 + 86.45 \cdot x_2$$

추정한 관계:

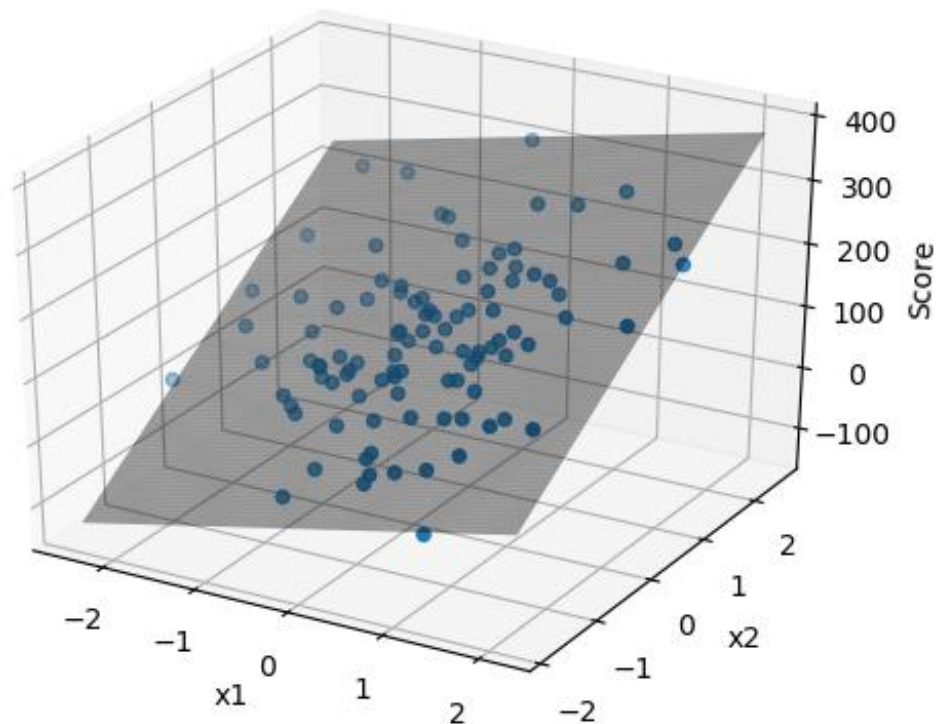
$$y = 100.73 + 31.31 \cdot x_1 + 83.68 \cdot x_2$$

X0 =

```
[[ 0.0465673  0.80186103]
 [-2.02220122  0.31563495]
 [-0.38405435 -0.3224172 ]
 [-1.31228341  0.35054598]
 [-0.88762896 -0.19183555]]
```

X =

```
[[ 1.    0.0465673  0.80186103]
 [ 1.   -2.02220122  0.31563495]
 [ 1.   -0.38405435 -0.3224172 ]
 [ 1.   -1.31228341  0.35054598]
 [ 1.   -0.88762896 -0.19183555]]
```



scikit-learn를 이용한 선형 회귀 (1)

```
from sklearn.linear_model import LinearRegression
from sklearn.datasets import make_regression
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits import mplot3d

# 데이터 생성
bias = 100
X0, y, w = make_regression(
    n_samples=100, n_features=2, bias=bias,
    noise=10, coef=True, random_state=1)
print("이론적 관계:")
print("y = %.2f + %.2f*x1 + %.2f*x2" % (bias, w[0], w[1]))

model = LinearRegression()
result = model.fit(X0, y)
print("추정한 관계:")
w = result.coef_
print("y = %.2f + %.2f*x1 + %.2f*x2" % (result.intercept_, w[0], w[1]))
```

feature augmentation 자동 처리

coef_ : 추정된 기울기(가중치) 벡터

intercept_ : 추정된 상수항

scikit-learn를 이용한 선형 회귀 (2)

```
X_new = [[0.012345, -0.104532], [-0.16539, 0.15843]]
yhat = result.predict(X_new)
print("{0}에 대한 예측치: {1}".format(X_new, yhat))

# 그래프 출력
X1_new = np.linspace(np.min(X0[:,0]), np.max(X0[:,0]), 100)
X2_new = np.linspace(np.min(X0[:,1]), np.max(X0[:,1]), 100)
m, n = np.meshgrid(X1_new, X2_new)

ax = plt.axes(projection='3d')
ax.set_xlabel('x1')
ax.set_ylabel('x2')
ax.set_zlabel('Score')
ax.scatter(X0[:,0], X0[:,1], y)

x1 = [i[0] for i in X_new]
x2 = [i[1] for i in X_new]
ax.scatter(x1, x2, yhat)

z = m*w[0] + n*w[1] + result.intercept_
ax.plot_surface(m, n, z, rstride=1, cstride=1, color='none', alpha=0.4)
plt.show()
```

scikit-learn를 이용한 선형 회귀 (3)

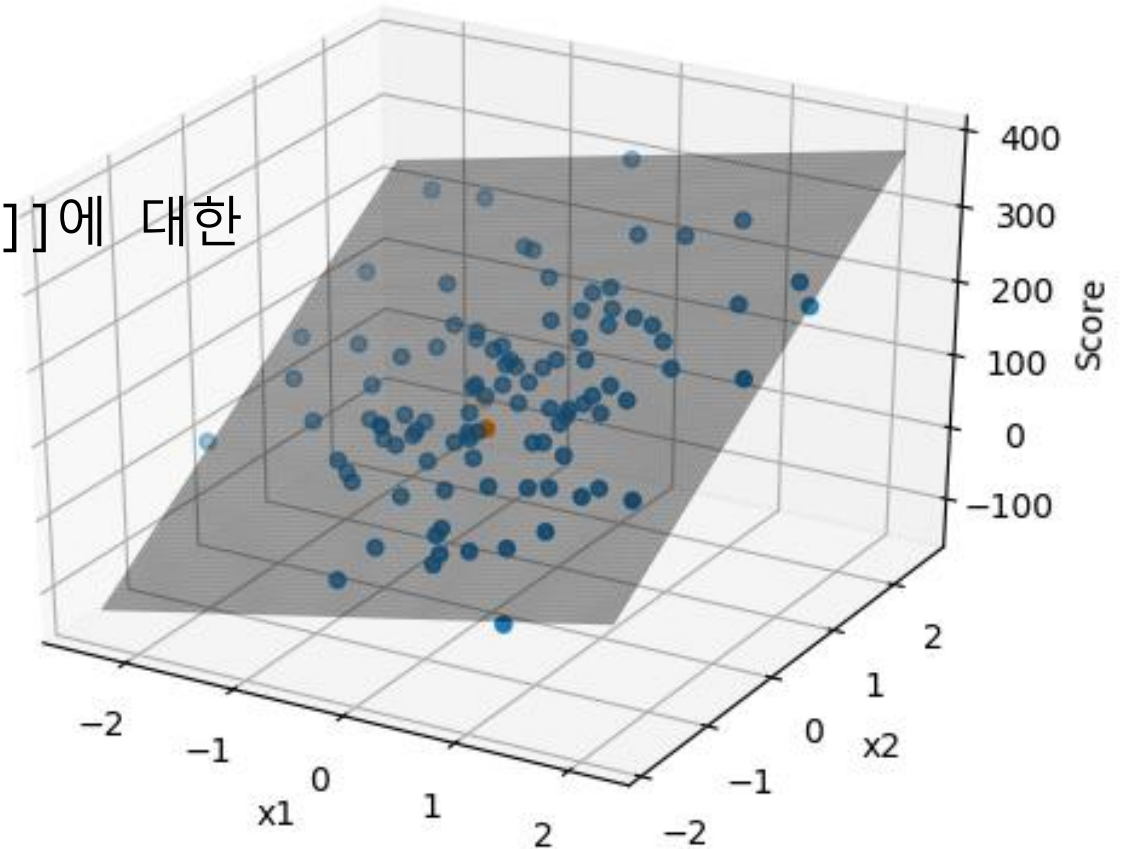
이론적 관계:

$$y = 100.00 + 32.27*x1 + 86.45*x2$$

추정한 관계:

$$y = 100.73 + 31.31*x1 + 83.68*x2$$

[[0.012345, -0.104532], [-0.16539, 0.15843]]에 대한
예측치: [92.36983533 108.80932525]



statsmodels를 이용한 선형 회귀 (1)

```
import statsmodels.api as sm
from sklearn.datasets import make_regression
import matplotlib.pyplot as plt
import numpy as np

bias = 100
X0, y, w = make_regression(
    n_samples=100, n_features=2, bias=bias,
    noise=10, coef=True, random_state=1)
X = sm.add_constant(X0)
print("이론적 관계:")
print("y = %.2f + %.2f*x1 + %.2f*x2" % (bias, w[0], w[1]))

model = sm.OLS(y, X)
result = model.fit()

print("추정한 관계:")
print("y = %.2f + %.2f*x1 + %.2f*x2" % (result.params[0],
result.params[1], result.params[2]))
print(result.summary())
```

feature augmentation 처리 필요

statsmodels를 이용한 선형 회귀 (2)

```
X = np.arange(X0.shape[0])
plt.plot(X, result.resid)
plt.title("residual vectors")
plt.xlabel("data #")
plt.show()

X_new = [[0.012345, -0.104532], [-0.16539, 0.15843]]
X_new = sm.add_constant(X_new)
yhat = result.predict(X_new)
print("{0}에 대한 예측치: {1}".format(X_new, yhat))
```

statsmodels를 이용한 선형 회귀 (3)

이론적 관계:

$$y = 100.00 + 32.27*x1 + 86.45*x2$$

추정한 관계:

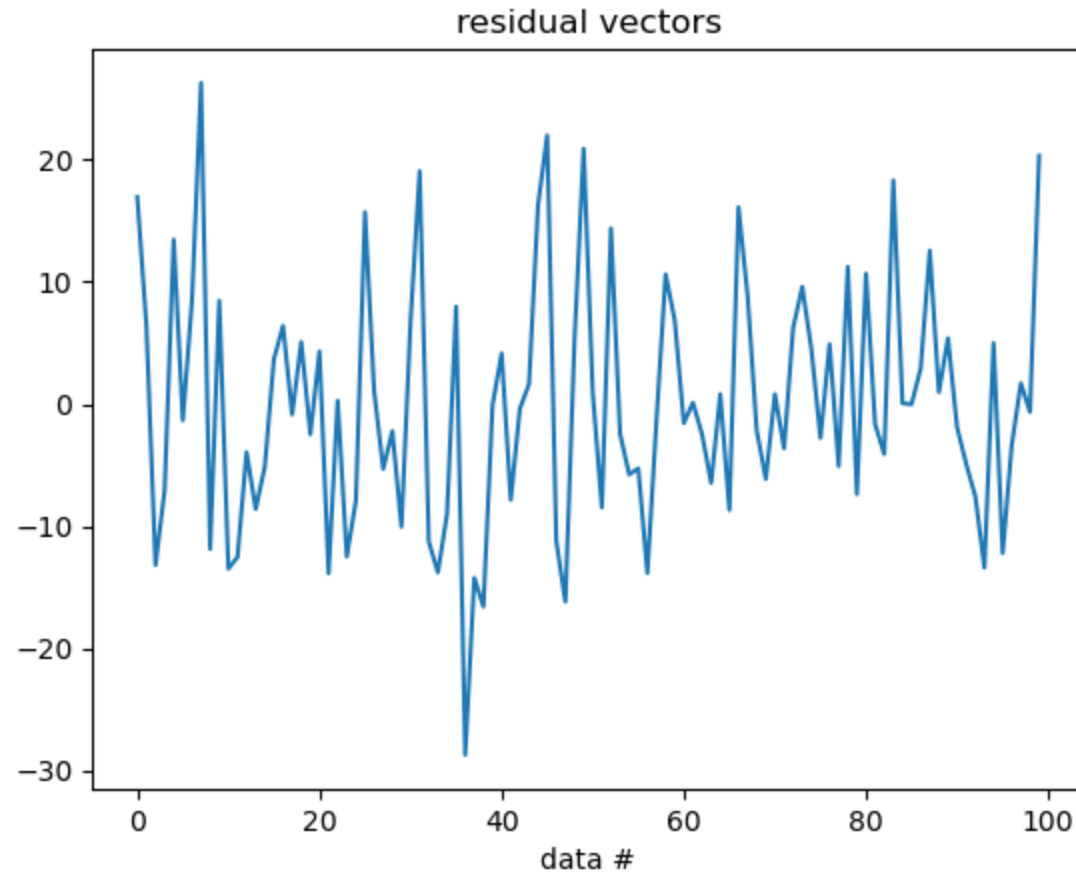
$$y = 100.73 + 31.31*x1 + 83.68*x2$$

$\begin{bmatrix} 1. & 0.012345 & -0.104532 \\ 1. & -0.16539 & 0.15843 \end{bmatrix}$ 에 대한 예측치: $[92.36983533 \ 108.80932525]$

statsmodels를 이용한 선형 회귀 (4)

```
=====
Dep. Variable:                y      R-squared:                0.982
Model:                        OLS     Adj. R-squared:           0.981
Method:                    Least Squares  F-statistic:            2620.
Date:                Thu, 17 Sep 2026  Prob (F-statistic):       3.84e-85
Time:                        07:49:36  Log-Likelihood:         -372.50
No. Observations:          100      AIC:                    751.0
Df Residuals:                97      BIC:                    758.8
Df Model:                      2
Covariance Type:            nonrobust
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
const          100.7304         1.054      95.564      0.000      98.638      102.822
x1              31.3091         1.091      28.690      0.000      29.143      33.475
x2              83.6783         1.198      69.846      0.000      81.301      86.056
=====
Omnibus:                1.279      Durbin-Watson:           1.824
Prob(Omnibus):           0.527      Jarque-Bera (JB):        1.065
Skew:                    0.253      Prob(JB):                0.587
Kurtosis:                2.999      Cond. No.                1.38
=====
```

statsmodels를 이용한 선형 회귀 (5)



statsmodels를 이용한 선형 회귀 (6)

```
print(result.params)  
print(result.resid.sum())
```

을 출력하는 경우

```
[100.73038617  31.30912998  83.67831905]  
1.050182163453428e-11
```

연습: 보스턴 집값 예측

- 보스턴 주택 가격 데이터를 사용하여 집 가격을 예측하는 회귀분석 수행

- sklearn 패키지에서 데이터를 제공

- 데이터를 불러오기와 특징 데이터 설명

- <https://datascienceschool.net/03%20machine%20learning/04.01%20%ED%9A%8C%EA%B7%80%E%B6%84%EC%84%9D%20%EC%98%88%EC%A0%9C.html#id2>

- 예측 모델 학습

- <https://wikidocs.net/49965>

- 실습

- 학습 데이터와 테스트 데이터의 예측 성능 결과 확인

- 특징 공학(feature engineering) 등을 통한 테스트 데이터의 예측 성능 개선

연습: 당뇨병 진행도 예측

- 당뇨병 데이터를 사용하여 당뇨병 진행도를 예측하는 회귀분석 수행

- sklearn 패키지에서 데이터를 제공

- 데이터를 불러오기와 특징 데이터 설명

- <https://datascienceschool.net/03%20machine%20learning/04.01%20%ED%9A%8C%EA%B7%80%EB%B6%84%EC%84%9D%20%EC%98%88%EC%A0%9C.html#id3>

- 실습

- 학습 데이터와 테스트 데이터의 예측 성능 결과 확인

- 특징 공학(feature engineering) 등을 통한 테스트 데이터의 예측 성능 개선

연습: 캘리포니아 주택 가격 예측

- 캘리포니아 주택 가격 데이터를 사용하여 집 가격을 예측하는 회귀분석 수행
 - sklearn 패키지에서 데이터를 제공
 - 데이터를 불러오기와 특징 데이터 설명
<https://wikidocs.net/49985>
 - 예측 모델 학습
<https://wikidocs.net/49986>
- 실습
 - 학습 데이터와 테스트 데이터의 예측 성능 결과 확인
 - 특징 공학(feature engineering) 등을 통한 테스트 데이터의 예측 성능 개선

연습: Kaggle 데이터세트 활용

- Kaggle 사이트에서 다변량 선형 회귀를 연습하기 위한 데이터 세트를 직접 선택하여 선형 회귀를 수행
 - 데이터세트 URL 및 선택 이유
 - 예측 목표
 - 특징(변수)에 대한 설명
 - 코드 및 결과 분석

- 선형 회귀분석의 기초

- <https://datascienceschool.net/view-notebook/58269d7f52bd49879965cdc4721da42d/>

- 선형 회귀 분석의 기초 (결정론적 모형)

- <https://datascienceschool.net/view-notebook/a0bf8d448ccf4165aef0c329670ada44/>

- 확률론적 선형 회귀모형

- <https://datascienceschool.net/view-notebook/743cdedec523447a907b2b0abda45533/>

- Ordinary Least Square (OLS)

- <https://datascienceschool.net/view-notebook/58269d7f52bd49879965cdc4721da42d/>

- statsmodels

- <https://www.statsmodels.org/stable/index.html>